

Práctica 2

Semáforos

1. Existen N personas que deben ser chequeadas por un detector de metales antes de poder ingresar al avión.

- a) **Analice el problema y defina qué procesos, recursos y semáforos/sincronizaciones serán necesarios/convenientes para resolverlo.**

Para que salga el avión, las personas deben pasar por el detector de metales, y a este detector de metales las personas van a pasar una a la vez.

Necesitaríamos un semáforo mutex para realizar exclusión mutua en el recurso compartido del problema, el detector de metales.

```
PROCESS PERSON
{
    while (true) {
        // Esperar para pasar por la aduana
        // Pasar por la aduana
        // Salir del control
    }
}
```

- b) **Implemente una solución que modele el acceso de las personas a un detector (es decir, si el detector está libre la persona lo puede utilizar; en caso contrario, debe esperar).**

sem mutex = 1;

```
PROCESS PERSON [1..N]
{
    while (true) {
        P(mutex);
        // Pasar por la aduana
        V(mutex);
    }
}
```

- c) **Modifique su solución para el caso que haya tres detectores.**

```
sem mutex = 3;
```

```
PROCESS PERSON [1..N]
{
    while (true) {
        P(mutex);
        // Pasar por la aduana
        V(mutex);
    }
}
```

¿Está bien esta solución? [Consultar](#)

- d) **Modifique la solución anterior para el caso en que cada persona pueda pasar más de una vez, siendo aleatoria esa cantidad de veces.**

```
sem mutex = 1;
```

```
PROCESS PERSONA [1..N]
{
    int veces = math.random(1, M)
    while (true) {
        for i:= 1 to veces do {
            P(mutex);
            // Pasar por la aduana
            V(mutex);
        }
    }
}
```

2. Un sistema de control cuenta con 4 procesos que realizan chequeos en forma colaborativa. Para ello, reciben el historial de fallos del día anterior (por simplicidad, de tamaño N). De cada fallo, se conoce su número de identificación (ID) y su nivel de gravedad (0=bajo, 1=intermedio, 2=alto, 3=crítico). Resuelva considerando las siguientes situaciones:

- a) Se debe imprimir en pantalla los ID de todos los errores críticos (no importa el orden)

```
int[] historial = ([N] ...)
```

```
PROCESS P[Id: 1..4]
{
    int rango = N/4;
    int pos_ini = rango * (Id - 1);
```

```

        for (i = 1; i <= rango; i++) {
            if (historial[i + pos_ini] == 3)
                print(i + pos_ini);
        }
    }
}

```

- b) Se debe calcular la cantidad de fallos por nivel de gravedad, debiendo quedar los resultados en un vector global

```

sem mutex = 1;
int[] historial = ...
int[] cant_fallos = ([4] 0)

PROCESS P[Id: 1..4]
{
    int rango = N/4;
    int pos_ini = rango * (Id - 1);

    for (i = 1; i <= rango; i++) {
        P(mutex);
        cant_fallos[historial[i + pos_ini]]++;
        V(mutex);
    }
}

```

Esta solución no es perfecta debido a que se podrían contar los fallos en un contador local, y luego sumar la cantidad obtenida al contador global. **Solución mejorada:**

```

sem se_fallo[4] = {1, 1, 1, 1}
int[] historial = ...
int[] cant_fallos = ([4] 0)

PROCESS P[Id: 1..4]
{
    int[] cant_fallos_local = ([4] 0);
    int rango = N/4;
    int pos_ini = rango * (Id - 1);

    for (i = 1; i <= rango; i++) {
        cant_fallos_local[historial[i + pos_ini]]++;
    }

    for (i = 1; i <= 4; i++) {
        P(se_fallo[i - 1]);
        cant_fallos[i - 1] += cant_fallos_local[i - 1];
        V(se_fallo[i - 1]);
    }
}

```

```
}
```

- c) Ídem b) pero cada proceso debe ocuparse de contar los fallos de un nivel de gravedad determinado

```
sem mutex = 1;  
int[] historial = ...  
int[] cant_fallos = ([4] 0)
```

```
PROCESS P[Id: 1..4]
```

```
{  
    int[] cant_fallos_local = ([4] 0);  
    int pos_ini = 0;  
  
    for (i = 1; i <= N; i++) {  
        if (historial[i] == N)  
            cant_fallos_local[historial[i]]++;  
    }  
  
    for (i = 1; i <= 4; i++) {  
        P(se_fallo[i - 1]);  
        cant_fallos[i - 1] += cant_fallos_local[i - 1];  
        V(se_fallo[i - 1]);  
    }  
}
```

Como el arreglo no se modifica nunca, si no que solo se lo lee por los cuatro procesos, no hace falta declarar sección crítica alguna.

3. Un sistema operativo mantiene 5 instancias de un recurso almacenadas en una cola. Además, existen P procesos que necesitan usar una instancia del recurso. Para eso, deben sacar la instancia de la cola antes de usarla. Una vez usada, la instancia debe ser encolada nuevamente para su reuso.

```
cola<recurso> cola = ([5] recurso);  
sem usar_recurso = 5;  
sem mutex = 1;
```

```
PROCESS P[1..N]
```

```
{  
    P(usar_recurso);  
    P(mutex); → No queremos conflictos a la hora de poppear la cola  
    recurso recurso_sacado = cola.pop();  
    V(mutex);  
    // El proceso utiliza el recurso
```

```

    P(mutex); → La instancia debe ser encolada nuevamente para su reuso
    cola.push(recurso_sacado);
    V(mutex);
    V(usar_recurso); → Permitir que otros procesos accedan a las instancias
}

```

¿Está bien esta solución? [Consultar](#)

4. Suponga que existe una BD que puede ser accedida por 6 usuarios como máximo al mismo tiempo. Además, los usuarios se clasifican como usuarios de prioridad alta y usuarios de prioridad baja. Por último, la BD tiene la siguiente restricción:

- no puede haber más de 4 usuarios con prioridad alta al mismo tiempo usando la BD.
- no puede haber más de 5 usuarios con prioridad baja al mismo tiempo usando la BD.

Indique si la solución presentada es la más adecuada. Justifique la respuesta.

Tenemos tres semáforos, uno que comparten todos los 6 procesos, otro que es para los 4 que son de prioridad alta, y el último para aquellos 5 de prioridad baja. Si entraran 6 procesos, no pueden entrar más hasta que salga uno, incluso si no se está utilizando al máximo. ¿Qué ocurre si entra un proceso de prioridad alta al “total” pero el semáforo “alta” ya está en 0? → Demora innecesaria

La solución presentada no es la más adecuada debido a que se produce demora innecesaria que se podría evitar. Para evitarla, simplemente hay que invertir el orden de las sentencias de los primeros semáforos.

5. En una empresa de logística de paquetes existe una sala de contenedores donde se preparan las entregas. Cada contenedor puede almacenar un paquete y la sala cuenta con capacidad para N contenedores. Resuelva considerando las siguientes situaciones:

- a) **La empresa cuenta con 2 empleados: un empleado Preparador que se ocupa de preparar los paquetes y dejarlos en los contenedores; un empleado Entregador que se ocupa de tomar los paquetes de los contenedores y realizar la entregas. Tanto el Preparador como el Entregador trabajan de a un paquete por vez.**

```
paq[] contenedores = ([N] null);  
int ocupado = 0; sem vacio = N;  
int libre = 0; sem lleno = 0;
```

```
PROCESS PREPARADOR  
{  
    while (true) {  
        paq paquete = new Paquete();  
        P(vacio);  
        contenedores[libre] = paquete;  
        libre = (libre + 1) mod N;  
        V(lleno);  
    }  
}
```

```
PROCESS ENTREGADOR  
{  
    while (true) {  
        paq tomado;  
        P(lleno);  
        tomado = contenedores[ocupado];  
        ocupado = (ocupado + 1) mod N;  
        V(vacio);  
        // Realizar entrega  
    }  
}
```

- b) **Modifique la solución a) para el caso en que haya P empleados Preparadores.**

```
paq[] contenedores = ([N] null);  
int ocupado = 0; sem vacio = N;  
int libre = 0; sem lleno = 0;  
sem mutex = 1;
```

```
PROCESS PREPARADOR
```

```

{
    while (true) {
        paq paquete = new Paquete();
        P(vacio);
        P(mutex);
        contenedores[libre] = paquete;
        V(mutex);
        libre = (libre + 1) mod N;
        V(lleno);
    }
}

PROCESS ENTREGADOR
{
    while (true) {
        paq tomado;
        P(lleno);
        tomado = contenedores[ocupado];
        ocupado = (ocupado + 1) mod N;
        V(vacio);
        // Realizar entrega
    }
}

```

c) Modifique la solución a) para el caso en que haya E empleados Entregadores.

```

paq[] contenedores = ([N] null);
int ocupado = 0; sem vacio = N;
int libre = 0; sem lleno = 0;
sem mutex = 1;

PROCESS PREPARADOR
{
    while (true) {
        paq paquete = new Paquete();
        P(vacio);
        contenedores[libre] = paquete;
        libre = (libre + 1) mod N;
        V(lleno);
    }
}

PROCESS ENTREGADOR
{
    while (true) {
        paq tomado;
        P(lleno);
        P(mutex);

```

```

        tomado = contenedores[ocupado];
        V(mutex);
        ocupado = (ocupado + 1) mod N;
        V(vacio);
        // Realizar entrega
    }
}

```

- d) **Modifique la solución a) para el caso en que haya P empleados Preparadores y E empleadores Entregadores.**

```

paq[] contenedores = ([N] null);
int ocupado = 0; sem vacio = N;
int libre = 0; sem lleno = 0;
sem mutexP = 1; sem mutexE = 1;

PROCESS PREPARADOR
{
    while (true) {
        paq paquete = new Paquete();
        P(vacio);
        P(mutexP);
        contenedores[libre] = paquete;
        V(mutexP);
        libre = (libre + 1) mod N;
        V(lleno);
    }
}

PROCESS ENTREGADOR
{
    while (true) {
        paq tomado;
        P(lleno);
        P(mutexE);
        tomado = contenedores[ocupado];
        V(mutexE);
        ocupado = (ocupado + 1) mod N;
        V(vacio);
        // Realizar entrega
    }
}

```

6. Existen N personas que deben imprimir un trabajo cada una. Resolver cada ítem usando semáforos:

- a) **Implemente una solución suponiendo que existe una única impresora compartida por todas las personas, y las mismas la deben usar de a una persona a la vez, sin importar el orden. Existe una función Imprimir(documento) llamada por la persona que simula el uso de la impresora. Sólo se deben usar los procesos que representan a las Personas.**

```
impresora imp = ...;  
sem mutex = 1;
```

```
PROCESS PERSONA[1..N]  
{  
    while (true) {  
        documento doc = cargarDocumento();  
        P(mutex);  
        imp.imprimir(doc);  
        V(mutex);  
    }  
}
```

- b) **Modifique la solución de (a) para el caso en que se deba respetar el orden de llegada.**

```
impresora imp = ...; boolean libre = true;  
sem mutex = 1; sem espera = ([N] 0)  
cola<Integer> cola_imprimir = new cola<Integer>();
```

```
PROCESS PERSONA[Id: 1..N]  
{  
    while (true) {  
        documento doc = cargarDocumento();  
        P(mutex);  
        if (libre)  
            libre = false; V(mutex);  
        else  
            cola_imprimir.push(Id); V(mutex); P(espera[Id]);  
  
        imp.imprimir(doc);  
  
        if (cola_imprimir.isEmpty())  
            libre = true;  
        else  
            int aux = cola_imprimir.pop(); V(espera[aux])  
    }  
}
```

- c) **Modifique la solución de (a) para el caso en que se deba respetar estrictamente el orden dado por el identificador del proceso (la persona X no puede usar la impresora hasta que no haya terminado de usarla la persona X-1).**

Para que esto funcione, tiene que existir un proceso que se encargue de activar o despertar al primer proceso, si no es imposible. La otra es utilizar la técnica passing the baton.

```
sem[] sem_persona = ([N] 0);
impresora imp = ...;
int indice = 1;
```

```
PROCESS PERSONA[Id: 1..N]
{
    documento doc = cargarDocumento();
    if (Id == 1)
        V(sem_persona[indice]);
    P(sem_persona[indice]);
    imp.imprimir(doc);
    if (indice <= N)
        indice++; V(sem_persona[indice]);
    V(sem_persona[Id]);
}
```

¿Está bien esta solución? [Consultar](#)

- d) **Modifique la solución de (b) para el caso en que además hay un proceso Coordinador que le indica a cada persona que es su turno de usar la impresora.**

De alguna manera la persona tiene que avisarle al coordinador que terminó de imprimir.

```
sem[] sem_persona = ([N] 0);
impresora imp = ...; sem mutex = 1;
int indice = 1; sem imprimiendo = 0;
cola<Integer> cola = ...; sem arribo = 0;
```

```
PROCESS PERSONA[Id: 1..N]
{
    documento doc = cargarDocumento();
    P(mutex);
    cola.push(Id);
    V(mutex);
    V(imprimiendo);
    P(espera[Id]);
    imp.imprimir(doc);
    V(arribo);
}
```

```
}
```

```
PROCESS COORDINADOR
```

```
{  
    while (True) {  
        int aux;  
        P(imprimiendo);  
        P(mutex);  
        aux = cola.pop();  
        V(mutex);  
        V(espera[aux]);  
        P(arribo);  
    }  
}
```

- e) **Modificar la solución (d) para el caso en que sean 5 impresoras. El coordinador le indica a la persona cuando puede usar una impresora, y cuál debe usar.**

Necesito una forma de decirle a la persona del Id que salió en la cola de que hay una impresora lista para usar. ¿Cómo lo hago?

```
sem[] espera_persona = ([N] 0);  
impresora[] impresoras = {imp, imp, imp, imp, imp}  
cola<Integer> cola_personas = new cola<Integer>();  
sem mutexCola = 1; sem mutex_id_imp = 1;  
sem llega = 0; sem imprimiendo = 0;  
int id_impresora = 0;
```

```
PROCESS COORDINADOR
```

```
{  
    while (True) {  
        P(llega);  
        P(mutexCola);  
        int id_actual = cola.pop();  
        V(mutexCola);  
        V(espera[id_actual]);  
        // Esperar a que la PERSONA imprima  
        P(imprimiendo);  
        P(mutex_id_imp);  
        id_impresora = (id_impresora + 1) mod 5;  
        V(mutex_id_imp);  
    }  
}
```

```
PROCESS PERSONA[Id: 1..N]
```

```
{  
    documento doc = cargarDocumento();  
    P(mutexCola);
```

```

cola.push(Id);
V(mutex_cola);
V(llega);
P(espera[Id]);
P(mutex_id_imp);
int id_imp = id_impresora; → Se le asigna una impresora
V(mutex_id_imp);
impresoras[id_imp].imprimir(doc);
V(imprimiendo); → Se terminó de imprimir
}

```

¿Está bien esta solución? [Consultar](#)

7. Suponga que se tiene un curso con 50 alumnos. Cada alumno debe realizar una tarea y existen 10 enunciados posibles. Una vez que todos los alumnos eligieron su tarea, comienzan a realizarla. Cada vez que un alumno termina su tarea, le avisa al profesor y se queda esperando el puntaje del grupo (depende de todos aquellos que comparten el mismo enunciado). Cuando un grupo terminó, el profesor les otorga un puntaje que representa el orden en que se terminó esa tarea de las 10 posibles.

```

cola<Integer> cola_terminaron = ...;
int[] puntaje_grupo = ([10] 0)
int[] cant_enunciados_tipo = ([10] 5)
int barrera = 0; sem sem_profe = 0;
sem mutex_barrera = 1; sem[] sem_grupo = ([5] 0);
sem mutex_cola = 1; sem sem_tarea_terminada = 0;
sem sem_barrera = 0; → No hace falta tener un semáforo privado para cada uno

```

```

PROCESS ALUMNO[Id: 1..50]
{
    int num_enunciado = elegir();
    P(mutex_barrera);
    barrera++;
    V(mutex_barrera);
    P(mutex_barrera);

    if (barrera == 50)
        V(sem_barrera); V(mutex_barrera);
    else
        P(sem_barrera); V(mutex_barrera);

    V(sem_profe);
}

```

```

// Realizar tarea...
P(mutexCola);
cola_terminaron.push(num_enunciado);
V(mutexCola);
V(sem_tarea_terminada);
P(sem_grupo[num_enunciado]); → Esperar a que termine todo el grupo
}

PROCESS PROFESOR
{
    int puntaje = 0;
    P(sem_profe);
    int id_alum_obtenido;
    int[] cant_grupo_enum = ([5] 0);

    for (i = 1; i <= 50; i++) → Hacer esto por y para cada proceso alumno
    {
        P(sem_tarea_terminada) → Esperar a que algún alumno termine
        P(mutexCola);
        id_alum_obtenido = cola_terminaron.pop()
        V(mutexCola);
        cant_grupo_enum[num_enunciado]++;
        if (cant_grupo_enum[num_enunciado] == 5)
        {
            puntaje_grupo[num_enunciado] = puntaje;
            V(sem_grupo[num_enunciado]); → Despertar grupo
            puntaje++; → Incrementar el puntaje para el próximo grupo
        }
    }
}

```

¿Está bien esta solución? [Consultar](#)

8. Una fábrica de piezas metálicas debe producir T piezas por día. Para eso, cuenta con E empleados que se ocupan de producir las piezas de a una por vez. La fábrica empieza a producir una vez que todos los empleados llegaron. Mientras haya piezas por fabricar, los empleados tomarán una y la realizarán. Cada empleado puede tardar distinto tiempo en fabricar una pieza. Al finalizar el día, se debe conocer cuál es el empleado que más piezas fabricó.

a) **Implemente una solución asumiendo que $T > E$**

```

int[] total_piezas = ([N] 0);
int piezas_restantes = T;
int barrera = 0;
sem[] espera = ([N] 0);
sem mutex = 1;
sem mutex_piez_res = 1;

PROCESS Empleado [Id: 1..E] {
    int cont_piezas = 0;
    P(mutex);
    barrera++;
    V(mutex);
    if (barrera == E) {
        for [i:=1 to T] {
            V(espera)
        }
    }
    P(espera);

    P(mutex_piez_res);
    while (piezas_restantes > 0) {
        piezas_restantes--;
        V(mutex_piez_res);
        // Realizar pieza
        cont_piezas++;
        P(mutex_piez_res);
    }
    V(mutex_piez_res);
    total_piezas[Id] = cont_piezas;
}

```

- b) **Implemente una solución que contemple cualquier valor de T y E.**
 La misma solución de arriba sirve para los otros dos casos

9. Resolver el funcionamiento en una fábrica de ventanas con 7 empleados (4 carpinteros, 1 vidriero y 2 armadores) que trabajan de la siguiente manera:

- Los carpinteros continuamente hacen marcos (cada marco es armando por un único carpintero) y los deja en un depósito con capacidad de almacenar 30 marcos.
- El vidriero continuamente hace vidrios y los deja en otro depósito con capacidad para 50 vidrios.
- Los armadores continuamente toman un marco y un vidrio (en ese orden) de los depósitos correspondientes y arman la ventana (cada ventana es armada por un único armador).

```
sem mutex_c = 1; sem mutex_a = 1
sem mutex_vid = 1; sem marcos = 0; sem vidrios = 0; sem deposito_m = 30;
sem deposito_vid = 50; cola<Marco> cola_m; cola<Vidrio> cola_vid;
cola<Ventana> cola_ven;
```

```
PROCESS Carpintero[1..4] {
    while (true) {
        marco aux_m;
        aux_m = crear_marco();
        P(deposito_m);
        P(mutex_c);
        cola_m.push(aux_m);
        V(mutex_c);
        V(marcos);
    }
}
```

```
PROCESS Vidriero {
    vidrio aux_vid;
    aux_vid = crear_vidrio();
    P(deposito_vid);
    P(mutex_vid);
    cola_vid.push(aux_vid);
    V(mutex_vid);
    V(vidrios);
}
```

```
PROCESS Armador[1..2] {
    while (true) {
        P(marcos);
        P(mutex_c)
        aux_m = cola_m.pop();
```

```

        V(deposito_m);
        V(mutex_c);

        P(vidrios);
        P(mutex_vid);
        aux_vid = cola_vid.pop();
        V(deposito_vid);
        V(mutex_vid);

        aux_ven = armar_ventana(aux_m, aux_vid);
        P(mutex_a);
        cola_ven.push(aux_ven);
        V(mutex_a);
    }
} → CONSULTAR

```

10. A una cerealera van T camiones a descargarse trigo y M camiones a descargar maíz. Sólo hay lugar para que 7 camiones a la vez descarguen, pero no pueden ser más de 5 del mismo tipo de cereal.

b) Implemente una solución que no use procesos adicionales (sólo camiones). No importa el orden de llegada para descargar. Nota: maximice la concurrencia.

```

sem sem_trigo = 5; sem sem_maiz = 5; sem sem_cerealera = 7;

PROCESS Camion_trigo[1..T] {
    P(sem_trigo); P(sem_cerealera); Descargar_trigo(); V(sem_cerealera);
    V(sem_maiz);
}

PROCESS Camion_maiz[1..M] {
    P(sem_maiz); P(sem_cerealera); Descargar_maiz(); V(sem_cerealera);
    V(sem_maiz);
}

```

11. En un vacunatorio hay un empleado de salud para vacunar a 50 personas. El empleado de salud atiende a las personas de acuerdo con el orden de llegada y de a 5 personas a la vez. Es decir, que cuando está libre debe esperar a que haya al menos 5 personas esperando, luego vacuna a las 5 primeras personas, y al terminar las deja ir para esperar por otras 5. Cuando ha atendido a las 50 personas el empleado de salud se retira. Nota: todos los procesos deben terminar su ejecución; suponga que el empleado tienen una función `VacunarPersona()` que simula que el empleado está vacunando a UNA persona.

```
sem sem_personas = 0;
sem mutexCola = 1;
sem[] sem_persona = ([N] 0);
```

```
PROCESS Empleado {
    for i:=1 to 10 do
        for i:=1 to 5 do
            P(sem_personas);
        for i:= 1 to 5 do
            P(mutexCola)
            int aux_id = cola_ids.pop();
            V(mutexCola);
            Vacunar(aux_id);
            V(sem_persona[aux_id]);
        }
}
```

```
PROCESS Persona[Id: 1..50] {
    P(mutexCola);
    cola_ids.push(Id);
    V(mutexCola);
    V(sem_personas);
    P(sem_persona[Id]);
}
```

12. Simular la atención en una Terminal de Micros que posee 3 puestos para hisopar a 150 pasajeros. En cada puesto hay una Enfermera que atiende a los pasajeros de acuerdo con el orden de llegada al mismo. Cuando llega un pasajero se dirige al Recepcionista, quien le indica qué puesto es el que tiene menos gente esperando. Luego se dirige al puesto y espera a que la enfermera correspondiente lo llame para hisoparlo. Finalmente, se retira.

Nota: suponga que existe una función Hisopar() que simula la atención del pasajero por parte de la enfermera correspondiente.

- a) **Implemente una solución considerando los procesos Pasajeros, Enfermera y Recepcionista.**

[CORREGIDO]

```
sem mutex_pr = 1; sem mutex_pg = 1; sem pacientes_gral = 0;
int personas_restantes = 150;
sem mutexCola_1 = 1; sem mutexCola_2 = 1; sem mutexCola_3 = 1;
cola ids_pacientes[1..3]; int[] pacientes_porCola = ([3] 0);
sem[] pacientes_enf[1..3]; sem[] pacientes_terminaron[1..150]
```

```
PROCESS Recepcionista {
    for (int i = 0; i < 150; i++)
        P(pacientes_gral);
        P(mutex_pg);
        int id_cli = id_general.pop();
        V(mutex_pg);
        → Podría controlar la cantidad con un vector de contadores
        cola_id = min(pacientes_porCola[1..3]);
        V(pacientes_enf[cola_id]);
        pacientes_porCola[cola_id]++;
        P(mutexCola_[cola_id]);
        ids_pacientes[cola_id].push(id_cli);
        V(mutexCola_[cola_id]);
    }
}
```

```
PROCESS Pasajero[Id: 1..50] {
    P(mutex_pg);
    id_general.push(Id);
    V(mutex_pg);
    V(pacientes_gral);
    P(pacientes_terminaron[Id]);
    P(mutex_pr);
    personas_restantes--;
    V(mutex_pr);
}
```

```
}
```

```
PROCESS Enfermera[Id: 1..3] {
```

```
    P(mutex_pr);
```

```
    while (personas_restantes > 0) {
```

```
        V(mutex_pr);
```

```
        P(pacientes_enf[Id]);
```

```
        if (not empty(ids_pacientes[Id]))
```

```
            P(mutexCola[Id]);
```

```
            int id_cli = ids_pacientes[id].pop();
```

```
            V(mutexCola[Id]);
```

```
            Hisopar(id_cli);
```

```
            V(pacientes_terminaron[id_cli]);
```

```
            pacientes_porCola[Id]--;
```

```
        }
```

```
    V(mutex_pr);
```

```
    V(pacientes_enf[1]); V(pacientes_enf[2]); V(pacientes_enf[3])
```

→ En realidad acá debería solo despertar a las otras dos enfermeras, pero no pasa nada si un semáforo recibe un punto que no usa

```
}
```

- b) **Modifique la solución anterior para que sólo haya procesos Pasajeros y Enfermera, siendo los pasajeros quienes determinan por su cuenta qué puesto tiene menos personas esperando.**

[CORREGIDO]

```
sem mutex_pr = 1; sem[] pacientes_enf = ([3] 0); sem[] mutexCola_enf = ([3] 1);
```

```
sem[] despertar = ([N] 0); int personas_restantes = 150;
```

```
cola<Integer>[] cola_enf = ([3] new Cola<Integer>());
```

```
int[] pacientes_porCola = ([3] 0);
```

```
PROCESS Enfermera[Id: 1..3] {
```

```
    P(mutex_pr);
```

```
    while (personas_restantes > 0) {
```

```
        V(mutex_pr);
```

```
        P(pacientes_enf[Id]);
```

```
        P(mutexCola_enf[Id]);
```

```
        if (not empty(cola_enf[Id]))
```

```
            int id_cli = cola_enf[Id].pop();
```

```
            V(mutexCola_enf[Id]);
```

```
            Hisopar(id_cli);
```

```
            V(despertar[id_cli]);
```

```
            pacientes_porCola[Id]--;
```

```
        P(mutex_pr); → Para mantener la coherencia
```

```
    }
```

```
    V(mutex_pr); → Para mantener la coherencia
```

```
}
```

```
PROCESS Pasajero[Id: 1..150] {
```

→ Controlar la cola menor con un vector de contadores, idem a arriba

```
cola_id = min(pacientes_por_cola[1..3]);
```

```
P(mutex_cola_enf[cola_id]);
```

```
cola_enf[cola_id].push(id);
```

```
V(mutex_cola_enf[cola_id]);
```

```
pacientes_por_cola[cola_id]++;
```

```
V(pacientes_enf[cola_id]);
```

```
P(despertar[id_cli]);
```

```
P(mutex_pr);
```

```
personas_restantes--;
```

```
V(mutex_pr);
```

```
}
```